Prepared for submission to JHEP

dSIBP Handbook

Contents

1 Scope and quick start 1

2 A complete mixed bubble-tree example 2

3 From SK rules to integral families 2

4 Integral representations 3

5 Massive function systems, derivatives, and EOM 3

6 Massless quotients and time IBP 4

7 Topology, scalar products, ISP, and symmetry 5

8 IBP generators and atomic operations 5

9 Seeds and continuous-index envelopes 6

10 Linear data and the Kira workflow 7

11 Initialization, messages, and inspection 8

12 Pure massive bubble closed loop 8

13 Tree vertex families 8

14 Common errors and current boundaries 9

15 Public API summary 9

1 Scope and quick start

dSIBP is a topology-driven Wolfram Language package for de Sitter Feynman integrals. It constructs time and loop-momentum IBP seeds, parameter-derivative operators, sector metadata, backend-neutral `linearData`, and Kira input files. It does not perform reduction internally. External reducers such as Kira or Rational Tracer remain separate steps controlled by the user.

The package supports arbitrary loop numbers and topologies, mixed massive and massless propagators, exact pinched-sector construction, EOM and symmetry canonicalization,

user-defined master bases, and differential-equation and scaling checks. The public topology schema is deliberately strict: package metadata is derived from the input graph rather than inferred from symbol names.

A standard session loads the package, initializes a family, generates symbolic seeds, expands continuous indices, and constructs linear data:

```
packageDir = FileNameJoin[{Directory[],"dSIBP"}];
If[!MemberQ[$Path,packageDir],AppendTo[$Path,packageDir]];
Needs["dSIBP`"];

context = DSInit[case];
seedData = DSSeeds[context];
allSeeds = DSAllSeeds[seedData];
ibpData = DSGenerateIBP[allSeeds,{-1,1}];
linearData = DSLinear[ibpData,context];
```

The package can be installed at any path; loading requires only the standard module tree and does not depend on the directory depth. Use `DSKiraPlan` and `DSKiraExport` when Kira input is required, run Kira outside the package, and use `DSKiraImport`, `DSDE`, and `DSScaleCheck` only after complete external results are available.

The bundled examples cover six complementary workflows:

- `01_mixed_bubble_workflow.wl`: basic initialization, generation, linearization, integral ordering, and planning;
- `02_function_system_hankel.wl`: Hankel and related function-system input;
- `03_single_massive_sunrise`: general two-loop seeds and parameter operators, without sampled relations or reduction;
- `04_pure_massive_bubble_closed_loop`: a 19-master Kira/DE/reference/scaling closed loop;
- `05_tree_two_vertex_time_ibp`: direct pure-time/tree relations and naive versus dlog differential equations;
- `06_mix_bubble_tree`: mixed cycle/bridge propagators, custom coordinates, pinched sectors, and an 84-master reduction summary.

2 A complete mixed bubble-tree example

A useful nontrivial input contains a massive cycle propagator and a massless cycle propagator between v_1 and v_2, followed by a massless fixed bridge from v_2 to v_3. Only the two cycle propagators carry loop-integral parity constraints. The bridge has no continuous loop-power slot and is excluded from parity.

Momentum declarations serve two distinct roles. `loopExternalMomenta` specifies independent directions entering the loop, while `independentExternalMomenta` specifies the

magnitudes of momenta that remain outside it. The package constructs the required Gram coordinates and checks whether these declarations are complete, overcomplete, or undercomplete. It never infers a role from a symbol's name.

Initialization returns the parsed topology, pinched-reachable sectors, canonical packs, zero points, normalization data, and capability reports. A typical high-level sequence is

```
context = DSInit[caseInput];
seedData = DSSeeds[context];
allSeeds = DSAllSeeds[seedData];
generated = DSGenerateIBP[allSeeds,
  {a[v1],-1,3},{a[v2],-1,3},{b[1],-2,3},{b[2],-2,3}];
linearData = DSLinear[generated,context];
```

The same example also demonstrates custom coordinates through `DSRedefineParameters`, fixed-bridge magnitudes, sector pinched maps, and exact round trips between user and internal scalar products.

3 From SK rules to integral families

3.1 Vertices and external phases

Each vertex is an association containing `id`, `vertexType`, and `externalLegEnergy`. The vertex type fixes the sign of the external phase, and the energy is the independent phase variable. The parser normalizes these signs once; all subsequent time derivatives and pinched rules use the same convention.

3.2 Propagator types

The propagator classes are massive time-ordered, massless time-ordered, massive Wightman, and massless Wightman, together with fixed lines. Massive time-ordered lines have a two-theta pinched system and a Hankel-type function space, while massless time-ordered lines use a two-vertex quotient representation. Wightman lines have no ordinary two-theta pinch channel. Fixed tree lines carry external magnitudes but no continuous loop-integral power.

A standard line uses `id`, `endpoints`, `momentum`, and `massType`. Massive lines additionally require `nu`. A custom massive function system may be supplied through `functionSystem`; no physical type is inferred from a mass parameter alone.

3.3 Pinched-reachable sectors

The root topology determines loop number, cycle/bridge roles, and propagator schemas. Pinched operations only pinch lines, merge vertex representatives, and update zero points and normalization. They do not create a new unrelated family or silently reduce the structural loop number.

Massive pinched terms are compiled from the Wronskian system. Simultaneous pinched terms merge a vertex only once, while integer shifts from all selected lines are accumulated. Uncut propagators are canonicalized in the target sector. Every reachable sector retains the metadata required by seeds, derivatives, and serializers.

4 Integral representations

In full mode the public integral head is

```
J[aList,linePacks,ispList]
```

The first slot contains compact vertex-time powers. Each unpinched-line pack has the form $\{b_e, n_{e,1}, n_{e,2}\}$, where b_e is a continuous denominator power and the two entries are discrete building-block states. A pinched line uses the corresponding compact bS representation. ISP powers are collected in the third slot and are measured from zero. Positive ISP powers are numerators; explicitly requested negative powers are also allowed.

In `ibpMode->"timeOnly"`, the public representation is instead

```
J[sectorKey,timeShifts,stateBits]
```

Here `sectorKey` is a fixed-width string of unpinched/pinched bits in root-line order. The other two slots contain compact time shifts and the sector-local discrete-state registry. The two schemas are intentionally distinct; no adapter silently rewrites full-loop Kira artifacts into the time-only representation.

5 Massive function systems, derivatives, and EOM

A massive building block is described by a 2×2 first-order system. Its derivative matrix and Wronskian determine, respectively, derivative shifts and pinched-pinch shifts. These two data channels are not interchangeable.

The `DSInit` option `NuConvention` selects the writing of every baseline formula that involves `nu`. The default `NuConvention->"Positive"` uses the input Hankel order as supplied; `NuConvention->"Negative"` flips its sign per massive line and byte-reproduces the baseline equations of earlier releases. The setting is frozen per line at initialization, and no later command accepts a convention option.

After a generator acts, any Hankel state of second order or higher is immediately reduced by the equation of motion. This happens before linear-system construction, so no symbolic $n \geq 2$ state is passed to a backend. Pinched shifts use the compiled Laurent coefficients and preserve the associated zero-point and prefactor changes.

Parameter derivatives act on both the integral and its explicit coefficients. The package also accounts for sector normalization, so a derivative of $J_s = N_s I_s$ includes the $D(\log N_s)$ term and, across a pinched, the appropriate N_s/N_t ratio.

6 Massless quotients and time IBP

Massless time-ordered propagators retain an ordered two-vertex three-slot pack. Regular and delta terms are handled in that same representation, including pinched pinch and coincident cancellation. Several time-ordered lines joining one vertex pair are treated by a common-theta construction rather than by independently duplicated pinched rules.

An odd subset of lines in a shared bundle determines the pinched transition. Its coefficient and target-sector mapping are evaluated once, and duplicate transitions are

removed by source, target, and selected-line identity. Massless fixed lines do not acquire a fictitious loop parity constraint.

Parallel massless time-ordered lines joining the same vertex pair are additionally folded by a built-in canonical layer. In the original per-line index representation the product kernel of such a bundle depends only on the total parity $\sum_e n_e \bmod 2$ of the endpoint states, so every member is mapped to the representative in which the first line in line order carries that parity and all remaining lines are zero; a member whose endpoint order is reversed contributes $(-1)^{n_e}$ per line. The fold sits after the single-line quotient and before symmetry in the canonical chain.

Seed enumeration distinguishes two cases. Without explicit parity constraints, or for a bundle no constraint touches, every member of the bundle is still enumerated over 00/10: one-sided states carry subsector boundary information that the fold only normalizes later. When a user constraint in `"parityConstraints"` touches a member of a bundle (banana total-parity sampling), initialization sums the constraints touching that bundle (including b indices, mod 2) into its total-parity constraint

$$\sum_{e\in\mathcal{B}} \big(b_e + n_{e,1} + n_{e,2}\big) \equiv r_{\mathcal{B}} \pmod 2, \tag{6.1}$$

and sampling enumerates only the parity slot $\{0, 1\}$ of the representative line (smallest line index), with the remaining members fixed to 0. `DSInit` prints a bilingual red notice listing which bundles are merged and the resulting total-parity target of each, then continues: it neither rejects nor stops. If only some members are constrained, the same rule sums the provided constraints and the resulting subsystem may leave the originally selected one; several parity subsystems may be selected simultaneously, and propagators without a parity selection are sampled over all parities.

A massless time-ordered line whose two endpoints coincide is converted to the pinched single-slot pack $\{b_{S,e}\}$, sharing all bookkeeping with pinched lines. This covers lines declared with equal `endpoints` and lines that become coincident when a pinched merges their vertices. At $\Delta = 0$ the kernel blocks take $M_0 = 2\theta(0) = 1$ and $M_1 = 0$, so the undifferentiated block is the constant 1, antisymmetric states vanish identically, and only the residual momentum-magnitude power survives. Declared self-loops convert at initialization with $b_{S0,e} = b_{0e}$ and no endpoint-state labels; the root graph audit still counts them as active cycle lines, since the residual b power carries the line's own loop momentum. In sector construction, coincident survivors join the pinched set of the sector, and distinct pinched routes that reach the same pinched set define a single sector entry, while pinched routing still follows each individual selection. Odd-state integrals are zeroed in the massless endpoint canonicalization against the source topology, and even-state pinched packs $\{b, 0, 0\}$ convert to the single slot automatically. A massive self-loop is not converted: it keeps its three-slot pack and the built-in reversal rule, and converted single-slot self-loops still enter the odd-ISP zeroing of that rule.

The direct pure-time/tree path uses the same physical pinched data but presents them in a sector-tagged public representation. Formula-based tree reduction is available only for

massive-only quotients whose derivation has been certified. In all other cases, the package returns a structured `PendingRederivation` result.

7 Topology, scalar products, ISP, and symmetry

The required topology input includes `vertices`, `lines`, `loopMomenta`, `loopExternalMomenta`, and `independentExternalMomenta`. ISP definitions are required when the propagators do not span all independent scalar products. Every ISP must be closed under the selected loop momentum generators.

The package distinguishes loop-external directions from loop-free magnitudes. Default loop coordinates are $\texttt{ssij} = \sqrt{\texttt{sp}[k_i, k_j]}$, with the derivative convention

$$\frac{\partial}{\partial \texttt{ssij}} = 2\,\texttt{ssij}\,\frac{\partial}{\partial \texttt{sp}[k_i, k_j]}. \tag{7.1}$$

Custom coordinates are introduced through `KinematicRules` or `DSRedefineParameters`. Undercomplete declarations stop initialization; overcomplete declarations may permit symbolic seed generation but disable unique derivatives and backend export.

Symmetry rules are applied as directed replacements. Users must choose one canonical representative for each equivalence class and provide oriented rules only. The symmetry of `sp` itself does not imply a graph symmetry: masses, energies, and momentum routing must also match.

8 IBP generators and atomic operations

Time generators differentiate with respect to conformal times. Momentum generators include loop scalings and the independent external directions required by the declared loop basis. The complete set is constructed before any index sampling.

The public atomic operations include `dtau`, `dqq`, `dqk`, and `ds`. The first three generate time and momentum IBP relations, while `ds` is the total parameter derivative used by the differential-equation workflow. Conversion helpers `rep2innerform` and `rep2outform` translate between user and internal coordinates without changing integral indices. `rep2Integrand` exposes an inert integrand for inspection; it does not integrate or generate a relation.

9 Seeds and continuous-index envelopes

9.1 Symbolic templates and shift metadata

`DSSeeds` enumerates the independent discrete templates before continuous sampling. Massive vertices use the four 0/1 states; massless quotients use their proved canonical representatives, with parallel bundles folded to the first-line parity form. EOM, symmetry, and sector canonicalization are applied while continuous indices remain symbolic.

`DSAllSeeds` returns the flat template list, while `DSSeedGroups` and `DSSeedGroupMetadata` expose the recommended grouping. `DSMetaSeedRange` records

the affine shift set of every continuous index. A flat input is treated as one group; each top-level element of a nested input is treated as a separate group.

A uniform target envelope is written as

```
ibp = DSGenerateIBP[allSeeds,{L,U}];
```

A detailed envelope assigns an interval to every root index. For an index with shift set Δ, the corresponding seed interval is the intersection of all inverse images:

$$x \in [L - \min \Delta,\ U - \max \Delta]. \tag{9.1}$$

For ISP indices, the lower endpoint is also bounded below by the user target lower endpoint. Empty inverse ranges are reported explicitly and disable the relevant completeness claim.

9.2 Seed-envelope cutting

`SeedEnvelopeCut` filters the inferred seed points before parity filtering and template substitution. The following calls disable cutting, use the default, and provide custom parameters:

```
ibp0 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->None];
ibp1 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->Automatic];
ibp2 = DSGenerateIBP[allSeeds,{-1,1},SeedEnvelopeCut->{10,4,5,4}];
```

Let C_+ be the sum of positive continuous slots, C_- the absolute sum of negative slots, and $T = C_+ + C_-$. For $C_+ \geq C_-$, acceptance requires $T \leq Ex_1$ and $C_- \leq Ex_2$; otherwise it requires $T \leq Ex_3$ and $C_+ \leq Ex_4$. For each group, let N_a be the number of a slots and N_b the number of b and bS slots, and set $N = \max(N_a, N_b)$. The default is $\{Ex_i\} = \{N+3, 4, 4, 4\}$. ISP indices are not counted in N. Every item of an explicit four-tuple may be an integer or `Automatic`, in which case only that default item is substituted.

The returned `seedEnvelopeCut` metadata records the resolved parameters, accepted and skipped point counts, and groups emptied by the cut. `None` restores the uncut point domain.

9.3 Canonical-support cutting

`TotalPowerLimit` is an independent filter on canonical equation support:

```
ibp3 = DSGenerateIBP[allSeeds,{-1,1},TotalPowerLimit->7];
ibp4 = DSGenerateIBP[allSeeds,{-1,1},
  SeedEnvelopeCut->{10,4,5,4},TotalPowerLimit->7];
```

For an integer m, the package first combines equal integrals and removes zero coefficients. It then retains an equation only if every canonical integral in its actual support lies in the per-slot target box and has continuous-index absolute sum at most m. The default `None` disables this filter. The two cut options can be used independently or together.

The full-mode fast path decomposes Association-based sealed templates into integral skeletons, integer shifts, and coefficient pieces, then memoizes canonical results by raw integral. Raw expression lists and failed templates use the original symbolic path. This changes performance, not the meaning of either cut option.

These options do not prove closure for an arbitrary target or basis. `completeSystemQ` describes generation completeness within the selected envelope and cutting policy. A new target, basis, or cut setting still requires an external reduction check that every requested target reduces to the requested masters.

10 Linear data and the Kira workflow

`DSLinear` converts generated relations into backend-neutral `linearData`. Its `integralList` is the authoritative backend order, and equations are deduplicated by their mathematical coefficients, constant terms, and nonlinear terms rather than by their provenance. `DSReorderIntegrals` explicitly changes that order before basis or backend construction. `DSUserMI` attaches a reversible ordered basis of single integrals or homogeneous linear combinations and constructs analytic derivative targets.

A formal Kira workflow has two stages: optional pre-reduction for basis exploration and a formal stage using the active basis and its minimal derivative closure. `DSKiraPlan` constructs the plan, and `DSKiraExport` serializes equations, maps, jobs, and manifest information without running Kira.

For a massive bubble, `numericStage->"postDerivative"` substitutes the declared parameter rules only after the analytic derivatives have been constructed; it does not necessarily make the reduction purely numerical. In the bundled example, `dim`, `nu`, `etaNu`, and `analyticRegulator` take exact rational values, but the kinematic variables remain symbolic during reconstruction. Kira therefore reconstructs in `dsc2` and `ip0`, and the resulting DE matrices remain symbolic in `ss11` and P_0. Comparison with the analytic reference at a fixed numerical point is a separate final check.

After external reduction, `DSKiraImport` checks manifest identity, target coverage, master order, and RHS closure. `DSDE` constructs matrices and sources in the active basis, and `DSScaleCheck` checks the applicable symbolic, fixed-section, or exact-point scaling certificate. A global residual master outside the requested target closure is not by itself a failure; acceptance is based on the requested targets and their RHS.

11 Initialization, messages, and inspection

`DSInit` validates topology, ISP closure, kinematics, and sector metadata. `DSInfo` returns initialization summaries. `DSKinematics` and `DSParameterNotation` expose coordinate proposals and current notation, while `DSRedefineParameters` performs a complete reinitialization under new rules.

`DSMessagesOn`, `DSMessagesOff`, and `DSMessagesQ` control optional information and warnings. Fatal errors remain visible. `AuditLevel` controls whether cached producer certificates are reused or full development audits are recomputed. `DSPublicAPI` returns the public function groups and the high-level options actually present in the loaded package.

12 Pure massive bubble closed loop

The bundled two-vertex massive bubble uses the $--$ branch, even parity on both cycle propagators, and equal vertex energies. Its target envelope is $a_v \in [0, 3]$ and $b_e \in [-3, 5]$, covering the 19-element basis and all derivative support. The example uses `SeedEnvelopeCut->Automatic` and `TotalPowerLimit->None`.

The current run generates 10952 IBP relations before linearization, followed by 2442 canonical relations, 2460 formal equations, and 1317 source integrals. Kira reduces all 300 requested targets, leaving none unreduced. The active basis has 19 masters; five additional global masters do not appear on the right-hand sides of the requested reductions. Import, 19×19 differential equations, and scaling pass 23 checks. The physical P_0, backend `ip0`, and physical k_s matrices each agree with the frozen analytic reference in all 361 entries.

The example requires an external workspace selected by `DSIBP_KIRA_WORKSPACE`; it must lie outside the package's Git repository (or, when the package is not inside a Git repository, outside the delivery directory), and the script refuses a workspace inside it. The package writes inputs but does not start Kira. After Kira finishes, `post_kira_check.wl` reads the existing results without regenerating inputs. Repository files retain only lightweight input and result summaries, not databases, reduction tables, caches, or runtime logs.

13 Tree vertex families

The tree workflow exposes `DSTreeNaiveIBP` and `DSTreeNaiveDE`. It constructs sector-tagged relations, ordered masters, and normalized differential equations from the same pinched and normalization metadata.

`DSTreeNaiveIBP` solves one-step raising relations in a specified basis. By default the ordered normalized masters are determined internally by the dlog master construction; an equivalent explicit list may also be supplied. `DSTreeNaiveDE` uses that reduction to construct the naive connection. For massless time-ordered quotients, tree reduction is available only after the required identities have been rederived on the quotient basis.

14 Common errors and current boundaries

Malformed topology, missing momentum directions, incomplete ISP closure, unknown or repeated range indices, symbolic discrete indices, and invalid cut specifications are reported as structured failures. Empty seed ranges are not silently ignored. Backend export also rejects unresolved internal coordinates, unsupported complex conventions, stale manifests, and incomplete reduction artifacts.

dSIBP does not run reduction internally. It does not infer physical symmetry from symbol names, and it does not turn a finite-envelope completeness claim into a proof for arbitrary targets. Naive tree reduction remains limited to the certified massive-only cases. Reference summaries record their evidence baseline and are not silently reused as proof for a different generation policy.

15 Public API summary

Function group	Purpose
`DSInit`, `DSInfo`, `DSKinematics`, `DSParameterNotation`, `DSRedefineParameters`	Initialization, topology and coordinate inspection, and redefinition.
`DSSeeds`, `DSAllSeeds`, `DSSeedGroups`, `DSSeedGroupMetadata`, `DSMetaSeedRange`	Symbolic discrete templates, grouping, and shift metadata.
`DSGenerateIBP`	Continuous expansion with `SeedEnvelopeCut` and `TotalPowerLimit`.
`DSLinear`, `DSReorderIntegrals`, `DSUserMI`	Backend-neutral equations, authoritative order, and active bases.
`DSKiraPlan`, `DSKiraExport`, `DSKiraImport`	Kira planning, serialization, and verified result import.
`DSDE`, `DSScaleCheck`	Differential equations and scaling certificates.
`DSTreeNaiveIBP`, `DSTreeNaiveDE`	Naive tree time-IBP reduction and differential equations.
`dtau`, `dqq`, `dqk`, `ds`	Atomic time, momentum, and parameter derivatives.
`rep2innerform`, `rep2outform`, `rep2Integrand`, `symmetry`, `repSymmetry0`	Coordinate conversion, inert integrands, and symmetry access.
`DSMessagesOn`, `DSMessagesOff`, `DSMessagesQ`, `DSPublicAPI`	Message control and public-interface inspection.

The exact options of a function are those returned by `Options[function]`. `DSGenerateIBP` exposes `SeedEnvelopeCut` (default `Automatic`) and `TotalPowerLimit` (default `None`), together with the shared audit and progress options.